Entrelíneas

Documentación del proyecto

Catálogo web, pedidos por WhatsApp, Mercado Pago, mostrador y caja

Proyecto de portfolio · Generado el 4 de octubre de 2026

Índice

1. Bitácora del proyecto
    - Estado actual
    - Entorno de desarrollo verificado
    - Sesiones
2. PRD — Requisitos del producto
    1. Resumen
    2. Problema
    3. Objetivos
    4. Usuarios
    5. Alcance del MVP
    6. Fuera del MVP (etapa 2 en adelante)
    7. Requisitos no funcionales
    8. Criterios de éxito de la demo
    9. Riesgos
3. Arquitectura
    1. Vista general
    2. Estructura del repositorio
    3. Modelo de datos
    4. Estados de una venta
    5. Flujos críticos
    6. Endpoints
    7. Autenticación
    8. Integraciones
    9. Variables de entorno
    10. Entornos y deploy
4. Reglas del proyecto
    1. Generales
    2. Idioma y nombres
    3. Backend (Express)
    4. Dinero, stock y caja (reglas críticas)
    5. Mercado Pago
    6. Frontend (Next.js)
    7. Seguridad
    8. Accesibilidad
    9. Tests
    10. Git
5. Tareas
    - Fase 0 — Preparación

Fase 1 — Base de datos y usuarios
Fase 2 — Libros y catálogo
Fase 3 — Carrito y pedidos
Fase 4 — Pagos
Fase 5 — Mostrador y caja
Fase 6 — Dashboard y facturas
Fase 7 — Calidad y publicación
Etapa 2 (después del MVP)

6. Memoria del proyecto
Contexto
Decisiones tomadas
Decisiones pendientes
Notas técnicas para recordar
Estado
Registro de sesiones

Bitácora del proyecto

Registro de avance, decisiones y observaciones. Se actualiza en cada sesión de trabajo y se regenera el PDF con `npm run pdf` dentro de `documentacion/`.

Estado actual

Ítem	Valor
Fase	0 — Preparación (sin empezar)
Código	Todavía no hay repositorio de código
Próximo paso	Decidir acceso a datos (Prisma o `mysql2`) y crear la estructura `apps/web` + `apps/api`

Entorno de desarrollo verificado

Herramienta	Versión
Node.js	20.19.5
npm	10.8.2
MySQL Server	9.3.0 (local)
Git	2.46.2
Sistema	Windows 11 Pro

Sesiones

2026-10-04 — Lectura de la documentación y armado del PDF

Qué se hizo

- Lectura completa de `PRD (1).md`, `arquitectura.md`, `rules.md`, `task.md` y `memory.md`.
- Se creó la carpeta `documentacion/` con un generador que une todos los documentos en un único PDF (`Entrelineas-Documentacion.pdf`).
- Se corrigió en `memory.md` una línea en blanco que partía la tabla de decisiones (las filas de Importación y Librería de Excel no se mostraban como tabla).

Observaciones sobre la documentación

1. **Falta `design.md`.** Lo citan `rules.md` (tokens de Tailwind), `task.md` (Fase 0) y `memory.md`. Hay que escribirlo antes de configurar Tailwind: paleta (papel, tinta, verde tela, ocre), tipografías (Literata + Public Sans) y el componente estante.

2. **Nombre del PRD.** El archivo se llama `PRD (1).md` pero los demás documentos lo citan como `PRD.md`. Conviene renombrarlo al copiarlo a `docs/`.
3. **Numeración en `arquitectura.md`.** En "Flujos críticos" la sección 5.5 (Importación) aparece antes que la 5.4 (Cierre de caja).
4. **Decisiones pendientes que bloquean:** acceso a datos (Prisma o `mysql2`), antes de la Fase 1; formato real del Excel de la librería, antes de la importación (Fase 2).

Pendiente

- Decidir Prisma o `mysql2`.
- Escribir `design.md`.
- Empezar la Fase 0.

PRD — Requisitos del producto

Documento de requisitos del producto. Define qué se construye, para quién y qué queda fuera del MVP.

1. Resumen

Entrelíneas es una librería ficticia de barrio que vende en su local y quiere vender online sin perder el trato personal con sus clientes. El sistema combina:

- Un **catálogo web público** donde cualquier persona arma un carrito sin crear cuenta.
- Un **checkout por WhatsApp**: el pedido se guarda en el sistema y se envía al WhatsApp de la librería, que lo confirma por chat.
- **Pagos** por Mercado Pago (Checkout Pro, confirmación automática) o transferencia (confirmación manual).
- Un **panel de administración** con pedidos, mostrador (punto de venta con lector de ISBN), caja diaria, carga de libros, usuarios y dashboard.

Es un proyecto de portfolio: tiene que funcionar de punta a punta en una demo pública, con datos de ejemplo y Mercado Pago en modo prueba.

2. Problema

Las librerías chicas venden por WhatsApp e Instagram de forma desordenada: el cliente pregunta si hay un libro, alguien revisa la estantería, se pasa el precio a mano, se cobra por transferencia y nadie sabe al final del día cuánto entró por cada medio. Las plataformas de e-commerce completas les resultan caras, impersonales y obligan al cliente a registrarse.

3. Objetivos

1. Que el cliente pueda ver el stock real y pedir en menos de un minuto, sin registrarse.
2. Que la librería centralice pedidos web y ventas del local en un solo sistema.
3. Que la caja del día muestre con exactitud cuánto entró por efectivo, transferencia y Mercado Pago, y por qué canal.
4. Que cargar un libro nuevo lleve segundos gracias al escáner y al autocompletado por ISBN.
5. (Portfolio) Demostrar integración real con Mercado Pago, manejo correcto de pagos (idempotencia, transacciones, anulaciones) y SEO con Next.js.

4. Usuarios

Usuario	Descripción	Acceso
Cliente	Persona que compra desde la web, generalmente desde el celular	Sin login
Vendedor	Atiende el mostrador, confirma pedidos, cobra, abre y cierra caja	Login, rol `vendedor`
Administrador	Dueño o encargado. Todo lo del vendedor más precios, usuarios, reportes y anulaciones	Login, rol `admin`

5. Alcance del MVP

5.1 Catálogo público

- Listado de libros con portada, título, autor, precio y disponibilidad.
- Búsqueda por título, autor e ISBN. Filtros por género y editorial. Paginación.
- Búsqueda y filtros reflejados en la URL (`/libros?genero=novela&q=borges`).
- Página de detalle por libro con URL amigable (`/libros/el-aleph-jorge-luis-borges`), sinopsis, datos y libros relacionados.
- Etiquetas Open Graph para que el link compartido por WhatsApp muestre portada, título y precio.
- Solo se muestran libros activos. Nunca se exponen datos internos.

5.2 Carrito y pedido por WhatsApp

- Carrito en el navegador. Datos del cliente: nombre y teléfono obligatorios, email opcional.
- Opción "Necesito factura" con CUIT/DNI, razón social y condición frente al IVA.
- Al confirmar: el pedido se guarda como `pendiente` con un código (`ENT-4821`) y un token público, y se abre WhatsApp con el mensaje armado.
- Protección antibots: Cloudflare Turnstile y límite de pedidos por IP.
- Los pedidos `pendiente` que nadie confirma vencen a las 2 horas (`expirada`).

5.3 Gestión de pedidos (admin)

- Lista con filtros por estado, búsqueda por código o teléfono.
- Confirmar pedido: descuenta stock y habilita el pago. Cancelar: devuelve el stock si ya se había descontado.
- Marcar como entregado o retirado.
- Copiar el link del pedido para enviarlo por WhatsApp.

5.4 Página pública del pedido

- URL única no adivinable (`/pedido/{token}`), sin login.
- Muestra estado, detalle y total.

- Si está confirmado: pagar con Mercado Pago, o ver los datos de transferencia (alias, CBU, monto exacto) y subir el comprobante.
- Si tiene factura cargada: botón "Descargar factura".

5.5 Pagos

- **Mercado Pago (Checkout Pro):** preferencia con `external_reference` = id de la venta; webhook que consulta el pago a la API y lo registra si está aprobado. Idempotente.
- **Transferencia:** número de operación obligatorio y único, comprobante opcional. Confirmación manual.
- **Efectivo, débito, crédito:** solo en el mostrador, registro manual.
- Un pago nunca se borra: se anula, registrando quién, cuándo y por qué.

5.6 Mostrador (punto de venta)

- Búsqueda por título o autor, o escaneo de ISBN (lector USB que funciona como teclado).
- Si el ISBN no existe, ofrecer darlo de alta.
- Cobro con uno o más métodos de pago. Stock descontado al registrar la venta.
- Requiere una caja abierta.

5.7 Caja diaria

- Apertura con efectivo inicial.
- Cada pago aprobado se asocia a la caja abierta en el momento del cobro, no a la fecha del pedido.
- Ingresos y egresos manuales (proveedor, insumos, retiro de efectivo).
- Cierre con arqueo: efectivo esperado contra contado, diferencia y observaciones.
- Resumen por método de pago y por canal. Historial de cajas cerradas.
- Los pagos de Mercado Pago que llegan sin caja abierta quedan "sin caja" y se asocian a la próxima caja que se abra.

5.8 Libros

- Alta escaneando el ISBN, con autocompletado desde Google Books y, si no hay resultado, Open Library.
- Portada copiada a Cloudinary (no se enlazan imágenes externas).
- Código interno para productos sin ISBN.
- Edición de precio, stock, género y estado (activo o inactivo).
- Cada cambio de stock queda registrado como movimiento.

5.9 Usuarios y roles

- Login con email y contraseña. Roles `admin` y `vendedor`.
- Cada venta, pago, movimiento y cierre de caja guarda el usuario que lo hizo.

5.10 Dashboard

- Ventas del día y del mes, por método de pago y por canal.
- Libros más vendidos. Pedidos pendientes de confirmar y de pago.

5.11 Facturas (carga manual)

- El admin sube el PDF de la factura emitida por la librería y lo asocia a la venta.
- La factura queda disponible en la página pública del pedido.
- La tabla de facturas ya contempla los campos de la futura integración con ARCA.

5.12 Importación desde Excel

- Pantalla del admin (solo rol `admin`) para subir el catálogo que la librería ya tiene en un archivo `.xlsx` o `.csv`.
- Plantilla descargable con las columnas recomendadas: ISBN, título, autor, editorial, género, precio, stock.
- Detección automática de columnas por el encabezado (por ejemplo "Título", "titulo" o "TITULO") y mapeo manual si no coincide.
- **Vista previa obligatoria** antes de guardar: cuántas filas se van a crear, cuántas actualizar, cuántas se omiten y cuáles tienen errores, con el motivo.
- Modo elegido explícitamente: solo crear libros nuevos, solo actualizar existentes, o ambos. Al actualizar, se elige qué campos se pisan (precio, stock, datos del libro).
- Los libros existentes se reconocen por ISBN o código interno.
- Cada cambio de stock por importación genera un movimiento de stock de tipo `importacion`.
- Informe final con descarga de las filas con error, para corregirlas y volver a importar.
- Opcional: completar datos faltantes y portadas desde Google Books / Open Library en segundo plano.
- Historial de importaciones: quién, cuándo, qué archivo y resultado.
- Script de línea de comandos con la misma lógica, para la carga inicial masiva o para cargar la demo.

6. Fuera del MVP (etapa 2 en adelante)

- Integración con ARCA en homologación (WSAA + WSFEv1, CAE, PDF con QR).
- QR de Mercado Pago integrado en el mostrador; Point integrado si hace falta.
- Encargos de libros sin stock.
- Historial de clientes por teléfono.
- Inventario avanzado: alertas de stock bajo, proveedores y compras.
- Promociones, descuentos y devoluciones.
- Exportación de reportes a Excel y PDF.
- Notificaciones por email.

7. Requisitos no funcionales

- **Mobile first** en el catálogo; el admin se usa principalmente en escritorio.
- **Rendimiento:** páginas del catálogo generadas en el servidor con revalidación; imágenes optimizadas por Cloudinary.
- **Seguridad:** contraseñas con bcrypt, sesión en cookie `httpOnly`, validación de entrada en todos los endpoints, firma del webhook validada, secretos solo en variables de entorno.
- **Integridad:** operaciones de dinero y stock dentro de transacciones; nada se borra en pagos, cajas ni ventas.
- **Accesibilidad:** contraste AA, foco visible, navegación por teclado (el mostrador debe poder usarse sin mouse).
- **SEO:** metadata por página, Open Graph, sitemap y URLs amigables.

8. Criterios de éxito de la demo

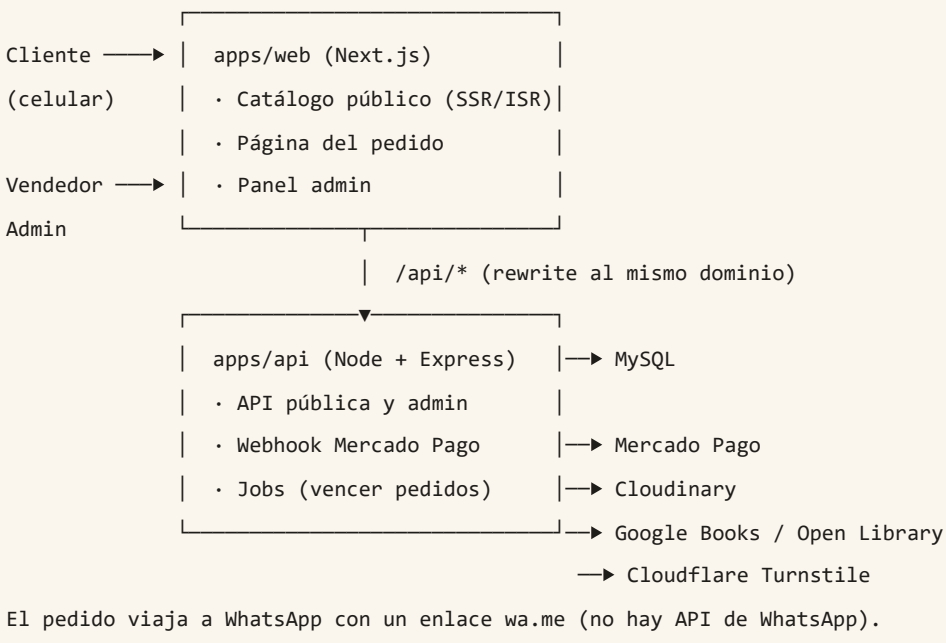
1. Un visitante pide un libro, paga con una tarjeta de prueba de Mercado Pago y el pedido aparece como pagado sin intervención manual.
2. Un vendedor abre caja, escanea un ISBN, cobra en efectivo y cierra caja con el arqueo correcto.
3. El link de un libro compartido por WhatsApp muestra la portada.
4. Los tests de caja, webhook y stock pasan.

9. Riesgos

Riesgo	Mitigación
Pedidos falsos o spam	Turnstile, límite por IP, confirmación manual por WhatsApp, vencimiento a 2 h
Webhook duplicado suma dos veces	Referencia única por pago y procesamiento idempotente
Venta simultánea del último ejemplar (web y mostrador)	Descuento de stock en transacción con verificación de cantidad
Cambios en APIs de Mercado Pago o de libros	Integraciones aisladas en servicios propios; revisar documentación en cada etapa
Excel del cliente desordenado (columnas distintas, ISBN convertidos a número, precios con formato)	Mapeo de columnas, normalización de datos, vista previa y descarga de errores
Alcance demasiado grande	MVP cerrado en este documento; todo lo demás va a etapa 2

Arquitectura

1. Vista general



El pedido viaja a WhatsApp con un enlace wa.me (no hay API de WhatsApp).

Decisiones principales

- **Next.js como frontend** (catálogo y admin) y **Express como API**. Express concentra toda la lógica de negocio, el acceso a datos y las integraciones. Next.js no accede a la base de datos.
- **Mismo dominio para web y API**: Next.js reescribe `/api/*` hacia Express (`rewrites` en `next.config.js`). Así la cookie de sesión es de primera parte y se evita configurar CORS con cookies entre dominios.
- **JavaScript** en ambos proyectos (coherente con el stack ya usado). Validación de datos con Zod.
- **Acceso a datos:** *decisión pendiente* entre Prisma y `mysql2` con SQL escrito a mano. El modelo de datos de este documento vale para cualquiera de las dos opciones. Toda consulta vive en la capa `repositories/`, así el resto del código no depende de esa decisión.

2. Estructura del repositorio

```
entrelineas/
├── apps/
│   ├── web/                      # Next.js (App Router)
│   │   ├── app/
│   │   │   ├── layout.js
│   │   │   ├── page.js           # Inicio
│   │   │   ├── libros/
│   │   │   │   ├── page.js       # Listado con búsqueda y filtros
│   │   │   │   └── [slug]/page.js
│   │   │   ├── carrito/page.js
│   │   │   ├── pedido/[token]/page.js
│   │   │   └── admin/
│   │   │       ├── layout.js     # Verifica sesión, menú lateral
│   │   │       ├── page.js       # Dashboard
│   │   │       ├── pedidos/
│   │   │       ├── mostrador/
│   │   │       ├── caja/
│   │   │       ├── libros/
│   │   │       └── usuarios/
│   │   ├── components/
│   │   ├── context/              # Carrito (client)
│   │   └── lib/                  # fetch a la API, formato de precios
│   └── api/                      # Express
│       └── src/
│           ├── routes/           # Definición de endpoints
│           ├── controllers/      # Leen request, llaman servicios, responden
│           ├── services/         # Lógica de negocio (caja, pagos, stock)
│           ├── repositories/     # Acceso a la base (Prisma o SQL)
│           ├── integrations/     # mercadopago, cloudinary, libros, turnstile
│           ├── middlewares/      # auth, roles, rate limit, errores
│           ├── validators/       # Esquemas Zod
│           ├── jobs/             # Vencimiento de pedidos, enriquecimiento de importaciones
│           ├── importacion/      # Lectura de Excel/CSV, normalización, validación, ejecución
│           ├── scripts/          # importar-excel.js (línea de comandos)
│           └── app.js
├── docs/                         # PRD, arquitectura, rules, design, task, memory
└── README.md
```

3. Modelo de datos

Montos en `DECIMAL(12,2)`. Fechas en UTC. Ninguna tabla de dinero se borra físicamente.

usuarios

Campo	Tipo	Notas
id	INT PK	
nombre	VARCHAR	
email	VARCHAR UNIQUE	
password_hash	VARCHAR	bcrypt
rol	ENUM('admin','vendedor')	
activo	BOOLEAN	
creado_en	DATETIME	

generos

`id`, `nombre`, `slug` UNIQUE.

libros

Campo	Tipo	Notas
id	INT PK	
isbn	VARCHAR(13) UNIQUE NULL	Se guarda como texto
codigo_interno	VARCHAR UNIQUE NULL	Productos sin ISBN
slug	VARCHAR UNIQUE	`titulo-autor`, sin tildes
titulo, autor, editorial	VARCHAR	
genero_id	FK generos	
sinopsis	TEXT NULL	
paginas	INT NULL	
precio	DECIMAL(12,2)	
stock	INT	Nunca negativo
portada_public_id	VARCHAR NULL	Id de Cloudinary
activo	BOOLEAN	
creado_en, actualizado_en	DATETIME	

ventas

Representa tanto un pedido web como una venta del mostrador.

Campo	Tipo	Notas
id	INT PK	

Campo	Tipo	Notas
codigo	VARCHAR UNIQUE	`ENT-4821`
token_publico	VARCHAR(32) UNIQUE NULL	Solo ventas web
canal	ENUM('web','local')	
estado	ENUM	Ver sección 4
cliente_nombre, cliente_telefono, cliente_email	VARCHAR NULL	
requiere_factura	BOOLEAN	
datos_fiscales	JSON NULL	cuit_dni, razon_social, condicion_iva
total	DECIMAL(12,2)	
usuario_id	FK usuarios NULL	Quién la registró o confirmó
stock_descontado	BOOLEAN	Para saber si cancelar devuelve stock
creado_en, confirmada_en, expira_en	DATETIME	

venta_items

`id`, `venta_id`, `libro_id`, `titulo` (copia), `precio_unitario` (copia), `cantidad`, `subtotal`. Los datos se copian para que un cambio de precio posterior no altere ventas pasadas.

cajas

Campo	Tipo	Notas
id	INT PK	
estado	ENUM('abierta','cerrada')	Una sola abierta a la vez
abierta_por, cerrada_por	FK usuarios	
abierta_en, cerrada_en	DATETIME	
efectivo_inicial	DECIMAL	
efectivo_esperado, efectivo_contado, diferencia	DECIMAL NULL	Se completan al cerrar
observaciones	TEXT NULL	

pagos

Campo	Tipo	Notas
id	INT PK	
venta_id	FK ventas	

Campo	Tipo	Notas
caja_id	FK cajas NULL	NULL = cobrado sin caja abierta
metodo	ENUM('efectivo','transferencia','mercadopago','debito','credito')	
monto	DECIMAL	
referencia	VARCHAR NULL	N.º de operación o id de pago de MP
comprobante_url	VARCHAR NULL	
estado	ENUM('pendiente','aprobado','rechazado','anulado')	
usuario_id	FK usuarios NULL	NULL si lo registró el webhook
anulado_por, anulado_en, motivo_anulacion		
creado_en	DATETIME	

Índice único: (`metodo`, `referencia`). Evita cargar dos veces la misma transferencia o el mismo pago de Mercado Pago.

movimientos_caja

`id`, `caja_id`, `tipo` ENUM('ingreso','egreso'), `concepto`, `monto`, `usuario_id`, `creado_en`, campos de anulación.

movimientos_stock

`id`, `libro_id`, `tipo` ENUM('alta','ingreso','venta','cancelacion','ajuste','importacion'), `cantidad` (con signo), `venta_id` NULL, `usuario_id` NULL, `motivo`, `creado_en`.

facturas

`id`, `venta_id`, `origen` ENUM('manual','arca'), `tipo` (A/B/C) NULL, `punto_venta` NULL, `numero` NULL, `cae` NULL, `cae_vencimiento` NULL, `pdf_url`, `usuario_id`, `creado_en`. Los campos de ARCA quedan vacíos en el MVP.

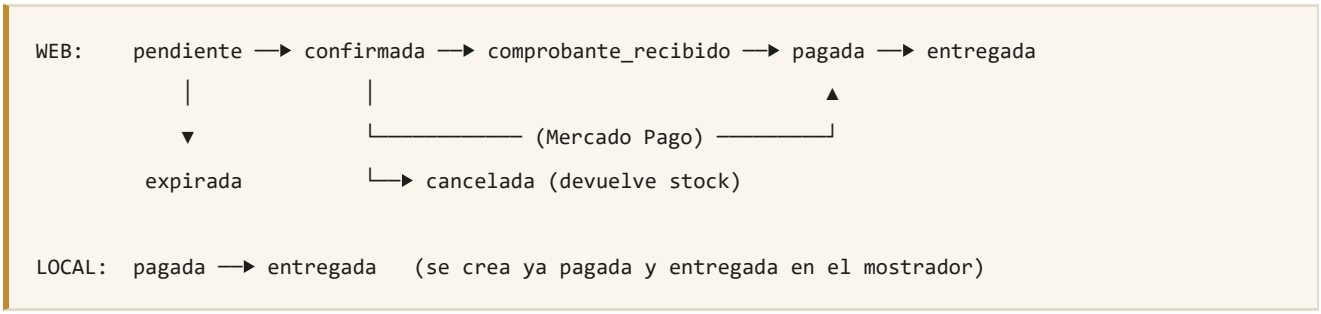
importaciones

`id`, `archivo_nombre`, `modo` ENUM('crear','actualizar','ambos'), `campos_actualizados` JSON, `estado` ENUM('vista_previa','procesando','completada','fallida'), `filas_total`, `creados`, `actualizados`, `omitidos`, `con_error`, `errores` JSON (fila, columna, motivo), `usuario_id`, `creado_en`, `finalizado_en`.

webhook_eventos

`id`, `proveedor`, `evento_id`, `payload` JSON, `procesado_en`, `error` NULL. Registro de auditoría de cada aviso recibido.

4. Estados de una venta



- `pendiente → expirada`: job cada 10 minutos para pedidos con `expira_en` vencido.
- `pendiente → confirmada`: acción del vendedor. Descuenta stock en transacción; si no alcanza, informa qué libros faltan.
- `confirmada → pagada`: webhook de Mercado Pago o confirmación manual de transferencia. Pasa a `pagada` cuando la suma de pagos aprobados cubre el total.

5. Flujos críticos

5.1 Pedido web

1. `POST /api/public/pedidos` con items, datos del cliente y token de Turnstile.
2. La API valida Turnstile, recalcula precios desde la base (nunca confía en precios del cliente), crea la venta `pendiente` con código, token y `expira_en`.
3. Devuelve el código y el texto del mensaje; el frontend abre `https://wa.me/{numero}?text=...`.

5.2 Pago con Mercado Pago

1. Desde `/pedido/{token}`, `POST /api/public/pedidos/{token}/mercadopago` crea la preferencia (`external_reference` = id de venta, `notification_url`, `back_urls`).
2. El cliente paga en Mercado Pago y vuelve a la página del pedido (solo informativo).
3. Mercado Pago llama a `POST /api/webhooks/mercadopago`. La API responde 200 de inmediato, valida la firma, guarda el evento y consulta el pago por id a la API de Mercado Pago.
4. Si está `approved`, en una transacción: verifica que no exista un pago con esa referencia, lo inserta en la caja abierta (o sin caja) y actualiza el estado de la venta.

5.3 Venta en el mostrador

1. El vendedor escanea o busca libros. `GET /api/admin/libros/isbn/{isbn}`.
2. `POST /api/admin/ventas` con items y pagos. En una transacción: verifica caja abierta y stock, crea venta `local`, items, pagos, movimientos de stock y descuenta stock.

5.5 Importación desde Excel

1. El admin sube el archivo: `POST /api/admin/importaciones/vista-previa` (máx. 5 MB, `.xlsx` o `.csv`, procesado en memoria, nunca se guarda en disco).
2. La API lee la primera hoja con ExcelJS, detecta columnas por encabezado y **normaliza** cada fila:

- ISBN leído como texto; se quitan guiones y espacios; se corrige el formato científico (`9.78950E+12`) y se convierte ISBN-10 a ISBN-13; se valida el dígito verificador.
- Precio: acepta `$15.000`, `15000`, `15.000,50`; se convierte a centavos.
- Stock: entero mayor o igual a cero.
- Texto: se recortan espacios y se unifica el género con los existentes.

3. Clasifica cada fila: `crear`, `actualizar`, `sin_cambios` o `error`, y detecta ISBN repetidos dentro del mismo archivo. Devuelve la vista previa y un id de importación en estado `vista_previa`.
4. El admin revisa, elige modo y campos a pisar y confirma: `POST /api/admin/importaciones/:id/confirmar`.
5. La API procesa en lotes de 200 filas, cada lote en una transacción: crea o actualiza libros, genera slugs y registra `movimientos_stock` de tipo `importacion`.
6. Guarda el resultado en `importaciones`. Si se pidió enriquecimiento, un job completa datos y portadas en segundo plano, respetando límites de las APIs externas.

5.4 Cierre de caja

`efectivo_esperado = efectivo_inicial + pagos en efectivo aprobados + ingresos manuales − egresos manuales`. Se guarda junto al contado y la diferencia. Una caja cerrada no admite cambios.

6. Endpoints

Públicos

Método	Ruta	Uso
GET	`/api/public/libros`	Listado con `q`, `genero`, `editorial`, `pagina`
GET	`/api/public/libros/:slug`	Detalle
GET	`/api/public/generos`	Filtros
POST	`/api/public/pedidos`	Crear pedido
GET	`/api/public/pedidos/:token`	Página del pedido
POST	`/api/public/pedidos/:token/mercadopago`	Crear preferencia
POST	`/api/public/pedidos/:token/comprobante`	Subir comprobante de transferencia
POST	`/api/webhooks/mercadopago`	Webhook

Admin (requieren sesión; marcados con ★ solo `admin`)

Método	Ruta	Uso
POST	`/api/auth/login`, `/api/auth/logout` · GET `/api/auth/me`	Sesión
GET, POST, PATCH	`/api/admin/libros`	ABM de libros (precio ★)

Método	Ruta	Uso
GET	`/api/admin/libros/isbn/:isbn`	Buscar en base o en APIs externas
GET	`/api/admin/pedidos`	Listado web
POST	`/api/admin/pedidos/:id/confirmar` · `/cancelar` · `/entregar`	Estados
POST	`/api/admin/ventas`	Venta de mostrador
POST	`/api/admin/pagos`	Registrar pago manual
POST	`/api/admin/pagos/:id/anular` ★	Anular pago
POST	`/api/admin/cajas/abrir` · GET `/api/admin/cajas/actual` · POST `/api/admin/cajas/cerrar`	Caja
GET	`/api/admin/cajas`	Historial
POST	`/api/admin/cajas/movimientos`	Ingreso o egreso manual
POST	`/api/admin/ventas/:id/factura`	Subir PDF
GET	`/api/admin/dashboard`	Métricas
GET	`/api/admin/importaciones/plantilla` ★	Descargar plantilla `.xlsx`
POST	`/api/admin/importaciones/vista-previa` ★	Subir archivo y obtener vista previa
POST	`/api/admin/importaciones/:id/confirmar` ★	Ejecutar la importación
GET	`/api/admin/importaciones` ★ · `/:id` · `/:id/errores`	Historial, detalle y descarga de errores
GET, POST, PATCH	`/api/admin/usuarios` ★	Usuarios

7. Autenticación

- Login devuelve una cookie `httpOnly`, `Secure`, `SameSite=Lax` con un JWT firmado (expira en 12 h).
- Middleware `requireAuth` y `requireRole('admin')` en Express.
- `app/admin/layout.js` llama a `/api/auth/me` y redirige a `/admin/login` si no hay sesión.

8. Integraciones

Servicio	Uso	Ubicación
Mercado Pago (SDK `mercadopago`)	Checkout Pro y consulta de pagos	`integrations/mercadopago.js`
Cloudinary	Portadas, comprobantes y facturas	`integrations/cloudinary.js`

Servicio	Uso	Ubicación
Google Books / Open Library	Datos por ISBN	`integrations/libros.js`
Cloudflare Turnstile	Captcha del pedido	`integrations/turnstile.js`
ExcelJS	Lectura de `.xlsx` / `.csv` y generación de plantilla e informe de errores	`importacion/lector.js`
WhatsApp	Enlace `wa.me` armado en el frontend	`apps/web/lib/whatsapp.js`

9. Variables de entorno

apps/api: `DATABASE_URL`, `JWT_SECRET`, `MP_ACCESS_TOKEN`, `MP_WEBHOOK_SECRET`, `PUBLIC_URL`, `CLOUDINARY_URL`, `GOOGLE_BOOKS_KEY`, `TURNSTILE_SECRET`, `WHATSAPP_NUMERO`, `TRANSFERENCIA_ALIAS`, `TRANSFERENCIA_CBU`.

apps/web: `API_URL`, `NEXT_PUBLIC_SITE_URL`, `NEXT_PUBLIC_TURNSTILE_SITE_KEY`, `NEXT_PUBLIC_WHATSAPP_NUMERO`, `NEXT_PUBLIC_CLOUDINARY_CLOUD`.

10. Entornos y deploy

- **Desarrollo:** MySQL local, API en `localhost:4000`, web en `localhost:3000`, túnel (ngrok o Cloudflare Tunnel) para el webhook.
- **Demo:** web en Vercel; API y MySQL en Railway o Render. Mercado Pago con credenciales de prueba.
- **Etapa 2 (ARCA):** módulo `integrations/arca/` con WSAA (token cacheado) y WSFEv1, contra homologación.

Reglas del proyecto

Reglas para cualquier persona o asistente de IA que escriba código en este repositorio. Ante una duda, priman este archivo, `arquitectura.md` y `PRD.md`, en ese orden.

1. Generales

- Leer `memory.md` y `task.md` antes de empezar una tarea, y actualizarlos al terminarla.
- Hacer solo lo que pide la tarea. Si aparece algo fuera del MVP, anotarlo en `task.md` (etapa 2) en vez de implementarlo.
- No agregar dependencias sin justificarlo en `memory.md`.
- Cambios chicos y verificables. Cada tarea termina con el código funcionando.

2. Idioma y nombres

- **Dominio en español** (coherente con el negocio): `venta`, `pago`, `caja`, `libro`, `obtenerVentaPorToken()`.
- Términos técnicos estándar en inglés cuando son de la librería o del framework (`req`, `res`, `router`, `middleware`).
- Variables y funciones en `camelCase`; componentes React en `PascalCase`; archivos de componentes `PascalCase.jsx`; resto en `kebab-case.js`.
- Tablas y columnas en `snake_case` y en plural para tablas.
- Textos de la interfaz en español rioplatense. Botones con verbo en infinitivo que dice exactamente qué pasa ("Agregar al carrito", "Enviar pedido por WhatsApp"); mensajes al usuario con voseo ("Revisá el teléfono").

3. Backend (Express)

- Capas: `routes → controllers → services → repositories`. Un controller no consulta la base; un repository no tiene lógica de negocio.
- Toda entrada se valida con Zod en `validators/` antes de llegar al controller.
- Errores con una clase `AppError(status, codigo, mensaje)` y un middleware central. Nunca devolver stack traces al cliente.
- Respuestas JSON con forma consistente: `{ data }` en éxito, `{ error: { codigo, mensaje } }` en error.
- Las integraciones externas viven solo en `integrations/`. Ningún service llama a `fetch` de un tercero directamente.

4. Dinero, stock y caja (reglas críticas)

- Montos como `DECIMAL(12,2)` en la base. En JavaScript, operar en **centavos enteros** (`Math.round(monto * 100)`) y convertir solo para mostrar. Nunca sumar floats directamente.
- **El precio lo decide el servidor.** Al crear un pedido o venta se ignoran precios que mande el cliente y se leen de la base.
- Toda operación que toque stock, pagos o caja va **dentro de una transacción**.
- El stock nunca queda negativo: el descuento se hace con una condición (`stock >= cantidad`) dentro de la transacción y se verifica cuántas filas se actualizaron.
- Cada cambio de stock genera un registro en `movimientos_stock`.
- **Nada se borra** en `ventas`, `pagos`, `cajas`, `movimientos_caja` ni `facturas`. Se anula con usuario, fecha y motivo.
- Una caja cerrada no se modifica.
- Un pago se asocia a la caja abierta **en el momento del cobro**.
- **Una importación nunca escribe sin vista previa confirmada**, y solo pisa los campos que el admin eligió.
- En importaciones, el ISBN siempre se lee y se guarda como **texto** (Excel suele convertirlo a número y romperlo).

5. Mercado Pago

- El webhook responde `200` de inmediato y procesa después.
- Validar la firma del webhook (`x-signature`) antes de procesar.
- Nunca usar el contenido del webhook como verdad: consultar siempre el pago por id a la API.
- Procesamiento idempotente: si ya existe un pago con esa referencia, no hacer nada.
- Las `back_urls` solo muestran mensajes; nunca cambian estados.
- Credenciales de prueba en la demo. El access token jamás llega al frontend.

6. Frontend (Next.js)

- Server Components por defecto. `"use client"` solo en componentes con estado, eventos o APIs del navegador.
- El catálogo se genera en el servidor con `revalidate`. `generateMetadata` en cada página de libro y de listado.
- Datos del admin: desde Client Components contra `/api/*` (la cookie viaja sola), o desde Server Components reenviando la cookie con `cookies()`.
- El carrito vive en un contexto de cliente y se guarda en `localStorage` envuelto en `try/catch`.
- Imágenes de portada siempre desde Cloudinary con transformaciones (`f_auto,q_auto,w_*`) y `alt` con el título del libro.
- Tailwind con los tokens definidos en `design.md`. No usar colores sueltos fuera de la paleta.

7. Seguridad

- Secretos solo en `.env` (nunca en el repo). Mantener `.env.example` actualizado sin valores reales.
- `bcrypt` con costo 12 para contraseñas.
- Rate limit en login y en creación de pedidos.
- Turnstile validado en el servidor.
- Tokens públicos de pedido generados con `crypto.randomBytes`, nunca con ids secuenciales.
- Endpoints públicos nunca devuelven datos internos (usuario, caja, costo, datos de otros pedidos).
- Archivos subidos: validar tipo y tamaño (imágenes y PDF, máximo 5 MB).

8. Accesibilidad

- Contraste AA mínimo, foco visible siempre, `label` en todos los campos.
- El mostrador se tiene que poder usar entero con teclado: el foco vuelve al campo del escáner después de cada acción.
- Respetar `prefers-reduced-motion`.

9. Tests

- Obligatorios para: cálculo de cierre de caja, procesamiento del webhook (incluido duplicado), descuento y devolución de stock, creación de pedido con precios manipulados.
- Vitest para unidades y Supertest para endpoints, contra una base de prueba.
- Los tests no llaman a servicios externos: se simulan las integraciones.

10. Git

- Ramas `feat/...`, `fix/...`, `docs/...`.
- Commits en español, en presente e imperativo: `agrega cierre de caja con arqueo`.
- Nunca commitear `.env`, claves ni certificados (`*.key`, `*.crt`).

Tareas

Marcar con `[x]` al terminar. Cada fase termina con algo que se puede probar. No pasar a la siguiente con la anterior rota.

Fase 0 — Preparación

- [ ] **Decidir acceso a datos: Prisma o `mysql2` con SQL.** Registrar la decisión en `memory.md`.
- [ ] Crear repositorio con `apps/web`, `apps/api` y `docs/` (copiar estos documentos).
- [ ] `apps/api`: Express, estructura de carpetas, middleware de errores, Zod, `.env.example`.
- [ ] `apps/web`: Next.js (App Router, JavaScript), Tailwind, fuentes Literata y Public Sans, tokens de `design.md`.
- [ ] Rewrite de `/api/*` hacia Express en `next.config.js`.
- [ ] MySQL local y conexión desde la API.
- [ ] Configurar Vitest y Supertest con base de prueba.

Fase 1 — Base de datos y usuarios

- [ ] Crear todas las tablas de `arquitectura.md` (migraciones).
- [ ] Script de datos de ejemplo (seed): 2 usuarios, 8 géneros, 60 libros con portadas.
- [ ] Login, logout y `/api/auth/me` con JWT en cookie `httpOnly`.
- [ ] Middlewares `requireAuth` y `requireRole`.
- [ ] Pantalla `/admin/login` y layout del admin con menú lateral.
- [ ] ABM de usuarios (solo admin).

Fase 2 — Libros y catálogo

- [ ] Integración Cloudinary (subir desde URL, armar URLs con transformaciones).
- [ ] Integración Google Books con respaldo de Open Library (`buscarDatosPorIsbn`).
- [ ] Admin: alta de libro escaneando ISBN, con autocompletado y revisión.
- [ ] Admin: listado, edición de precio y stock (con movimiento de stock), activar y desactivar.
- [ ] Generación de slug único.
- [ ] Endpoints públicos de libros y géneros (sin datos internos).
- [ ] Página `/libros` con búsqueda, filtros en la URL y paginación.
- [ ] Componente estante (portadas alineadas abajo sobre la línea ocre).
- [ ] Página `/libros/[slug]` con `generateMetadata` y Open Graph.
- [ ] Página de inicio con estante de novedades y géneros.
- [ ] `sitemap.js` y `robots.js`.
- [ ] **Importación desde Excel:**

- [ ] Conseguir (o simular) el Excel real de la librería y anotar su formato en `memory.md`.
- [ ] Módulo `importacion/`: lector (ExcelJS), detección de columnas, normalización (ISBN, precio, stock) y validación.
- [ ] Tests de normalización: ISBN con guiones, en formato científico, ISBN-10, precio con `$` y separadores, filas vacías, ISBN repetidos.
- [ ] Tabla `importaciones` y tipo `importacion` en `movimientos_stock`.
- [ ] Endpoints de plantilla, vista previa, confirmación, historial y descarga de errores.
- [ ] Pantalla `/admin/libros/importar`: subir archivo, mapear columnas, vista previa, elegir modo y campos, confirmar, ver informe.
- [ ] Procesamiento por lotes en transacciones.
- [ ] Job opcional de enriquecimiento (datos y portadas desde APIs de libros).
- [ ] Script `scripts/importar-excel.js` para línea de comandos (usarlo para cargar la demo).

- [ ] Probar la vista previa del link en WhatsApp.

Fase 3 — Carrito y pedidos

- [ ] Contexto de carrito con persistencia en `localStorage`.
- [ ] Página `/carrito` con datos del cliente y opción de factura.
- [ ] Cloudflare Turnstile en el frontend y validación en la API.
- [ ] `POST /api/public/pedidos`: precios desde la base, código `ENT-XXXX`, token, vencimiento.
- [ ] Armado del mensaje y apertura de `wa.me`.
- [ ] Rate limit en creación de pedidos.
- [ ] Job de vencimiento de pedidos pendientes.
- [ ] Admin: listado de pedidos con filtros y búsqueda.
- [ ] Confirmar (descuenta stock en transacción), cancelar (devuelve stock), entregar.
- [ ] Página pública `/pedido/[token]` con estado y línea de pasos.
- [ ] Test: pedido con precios manipulados usa los de la base.
- [ ] Test: confirmar sin stock suficiente falla sin descontar nada.

Fase 4 — Pagos

- [ ] Integración Mercado Pago: crear preferencia y consultar pago.
- [ ] Botón "Pagar con Mercado Pago" en la página del pedido.
- [ ] Webhook: respuesta inmediata, validación de firma, registro en `webhook_eventos`, consulta del pago, registro idempotente.
- [ ] Probar con túnel (ngrok o Cloudflare Tunnel) y tarjetas de prueba.
- [ ] Transferencia: datos de pago en la página del pedido y subida de comprobante.
- [ ] Admin: registrar pago por transferencia con número de operación único.
- [ ] Anulación de pagos (solo admin, con motivo).
- [ ] Cambio automático de estado a `pagada` cuando los pagos cubren el total.

- [ ] Test: webhook duplicado registra un solo pago.
- [ ] Test: transferencia con número de operación repetido es rechazada.

Fase 5 — Mostrador y caja

- [ ] Abrir caja con efectivo inicial (una sola abierta a la vez).
- [ ] Indicador de caja en el encabezado del admin.
- [ ] Mostrador: campo de escaneo con foco permanente, búsqueda por texto, alta rápida si el ISBN no existe.
- [ ] Cobro con uno o varios métodos y cálculo de vuelto.
- [ ] `POST /api/admin/ventas` en una transacción (venta, items, pagos, stock, movimientos).
- [ ] Atajos de teclado del mostrador.
- [ ] Ingresos y egresos manuales.
- [ ] Asociar a la caja que se abre los pagos web que llegaron sin caja.
- [ ] Cierre con arqueo y diferencia calculada en vivo.
- [ ] Resumen por método y por canal; historial de cajas.
- [ ] Test: cálculo de efectivo esperado con ventas, ingresos, egresos y pagos anulados.
- [ ] Test: no se puede modificar una caja cerrada.

Fase 6 — Dashboard y facturas

- [ ] `GET /api/admin/dashboard` con ventas del día y del mes, métodos, canales, más vendidos y pendientes.
- [ ] Pantalla de inicio del admin.
- [ ] Subida de PDF de factura a una venta.
- [ ] Botón "Descargar factura" en la página pública del pedido.

Fase 7 — Calidad y publicación

- [ ] Revisión de accesibilidad (teclado, contraste, foco, `alt`).
- [ ] Revisión de seguridad con la lista de `rules.md`.
- [ ] Deploy de la web en Vercel y de la API con MySQL en Railway o Render.
- [ ] Webhook de Mercado Pago apuntando a la URL de producción de la demo.
- [ ] Usuario de demo y datos de ejemplo cargados.
- [ ] README: problema, capturas o GIF, stack, diagrama del flujo, cómo correrlo, credenciales de demo, tarjeta de prueba, decisiones técnicas y próximas funcionalidades.

Etapa 2 (después del MVP)

- [ ] ARCA en homologación: certificado (WSASS), WSAA con token cacheado, WSFEv1, CAE, PDF con QR.

- [ ] QR de Mercado Pago integrado en el mostrador.
- [ ] Point integrado (si hace falta).
- [ ] Encargos de libros sin stock.
- [ ] Historial de clientes por teléfono.
- [ ] Alertas de stock bajo, proveedores y compras.
- [ ] Promociones, descuentos y devoluciones.
- [ ] Exportar reportes a Excel y PDF.
- [ ] Notificaciones por email.

Memoria del proyecto

Contexto acumulado para retomar el trabajo (persona o asistente de IA). Leer al empezar cada sesión. Actualizar al terminar: decisiones nuevas, problemas encontrados y estado.

Contexto

- Proyecto de **portfolio**. La librería "Entrelíneas" es ficticia.
- Objetivo de la demo: que quien la revise pueda pedir, pagar con Mercado Pago de prueba y usar el mostrador y la caja en pocos minutos.
- Documentos de referencia: `PRD.md` (qué), `arquitectura.md` (cómo), `rules.md` (reglas), `design.md` (visual), `task.md` (avance).

Decisiones tomadas

Tema	Decisión	Motivo
Login de clientes	No hay. El pedido se valida por WhatsApp con un código	Menos fricción; el número de WhatsApp identifica al cliente
Checkout	Pedido guardado como `pendiente` + enlace `wa.me`	No requiere la API de WhatsApp Business
Antibots	Cloudflare Turnstile + rate limit + vencimiento a 2 h	Proteger sin pedir cuenta
Pago online	Mercado Pago Checkout Pro con webhook	Confirmación automática; no se manejan tarjetas
Transferencias	Número de operación único + comprobante; confirmación manual	No se pueden verificar automáticamente
Ventas del local	Misma tabla `ventas` con `canal = 'local'`	Una sola fuente para la caja
Pagos en la caja	Cuentan en la caja abierta al momento del cobro	La caja refleja lo cobrado ese día
Borrados	Ningún pago, venta, caja ni factura se borra; se anula	Historial confiable
Escáner	Lector USB como teclado, ISBN-13 como identificador	Sin drivers ni librerías
Datos de libros	Google Books, con Open Library de respaldo	Mejor cobertura en español
Imágenes	Cloudinary; se guarda `public_id`, no la URL	Transformaciones y cambio de proveedor fácil

Tema	Decisión	Motivo
Frontend	Next.js (App Router)	SEO y vista previa de links en WhatsApp
Backend	Express + MySQL	Stack ya conocido
Web y API	Mismo dominio vía rewrites de Next	Cookies de sesión sin CORS entre dominios
Lenguaje	JavaScript	Coherencia con proyectos anteriores
Facturas	MVP: carga manual de PDF. Etapa 2: ARCA en homologación	Terminar el MVP primero
QR y Point de Mercado Pago	MVP: registro manual. Etapa 2: integración	Reutilizar la lógica del webhook después
Diseño	Paleta papel, tinta, verde tela y ocre; Literata + Public Sans; catálogo como estante	Identidad propia de librería
Importación desde Excel	En el MVP, con vista previa obligatoria y modo explícito (crear, actualizar o ambos)	La librería ya tiene su catálogo en Excel; nada se pisa sin que el admin lo vea
Librería de Excel	ExcelJS (`.xlsx` y `.csv`); los `.xls` viejos se guardan como `.xlsx` antes de subir	Mantenida y disponible en npm; el paquete `xlsx` de npm quedó desactualizado

Decisiones pendientes

- **Acceso a datos: Prisma o `mysql2` con SQL.** Definir antes de la fase 1.
- **Formato del Excel de la librería:** qué columnas tiene, cómo vienen los precios y si incluye stock. Ajustar el mapeo automático cuando se conozca.

Notas técnicas para recordar

- Mercado Pago: nunca confiar en el cuerpo del webhook ni en las `back_urls`; consultar el pago por id.
- Para probar el webhook en local hace falta un túnel; la URL cambia en cada ejecución del plan gratuito, así que va en `.env`.
- ARCA (etapa 2): el WSASS se adhiere con la clave fiscal de persona física desde el Administrador de Relaciones; el token del WSAA se cachea hasta su vencimiento.
- Revisar la documentación vigente de Mercado Pago y de ARCA al empezar cada integración: sus APIs cambian.

Estado

- Fase actual: **0 — Preparación.**
- Última actualización: planificación inicial del proyecto y documentación.

Registro de sesiones